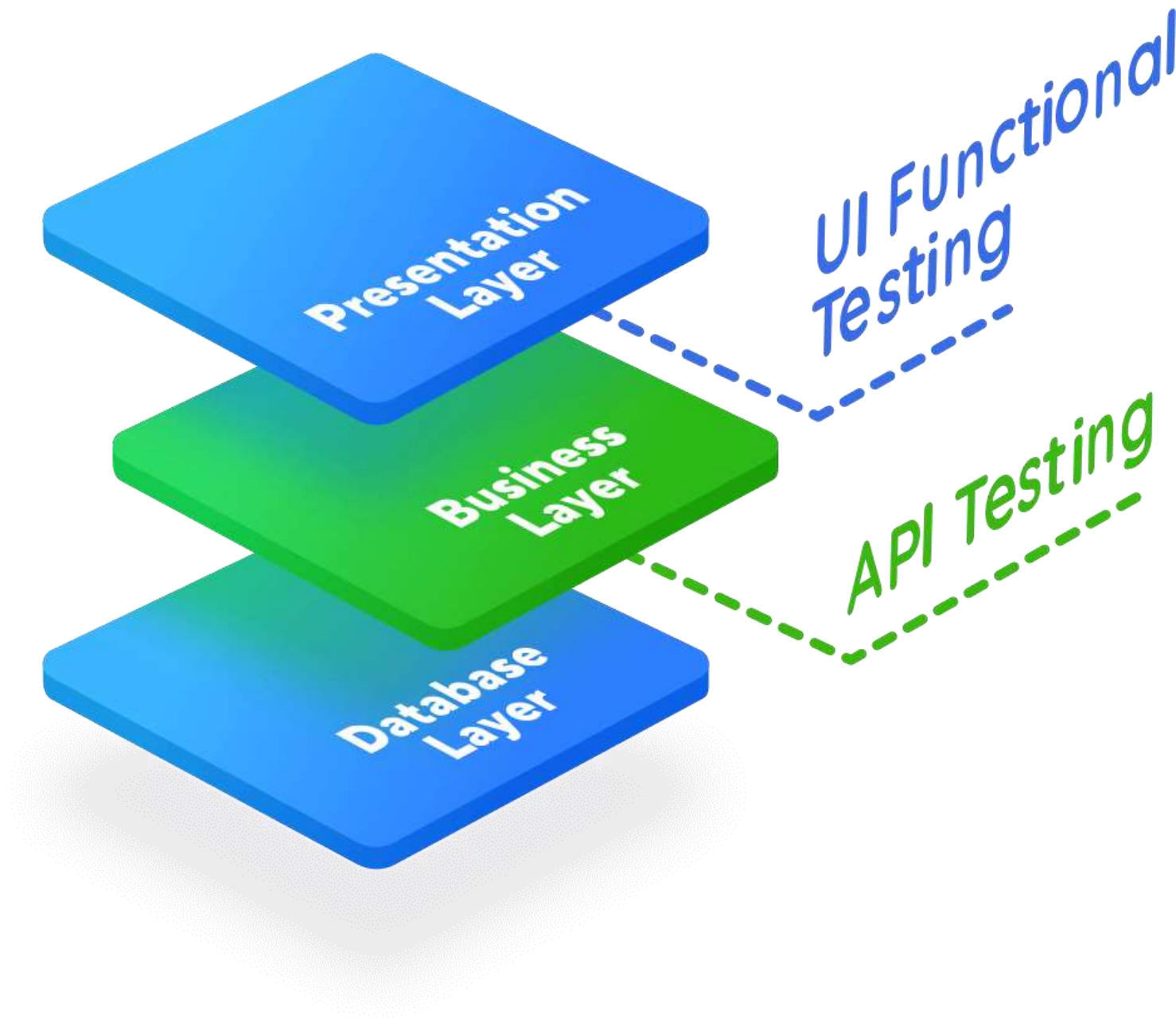




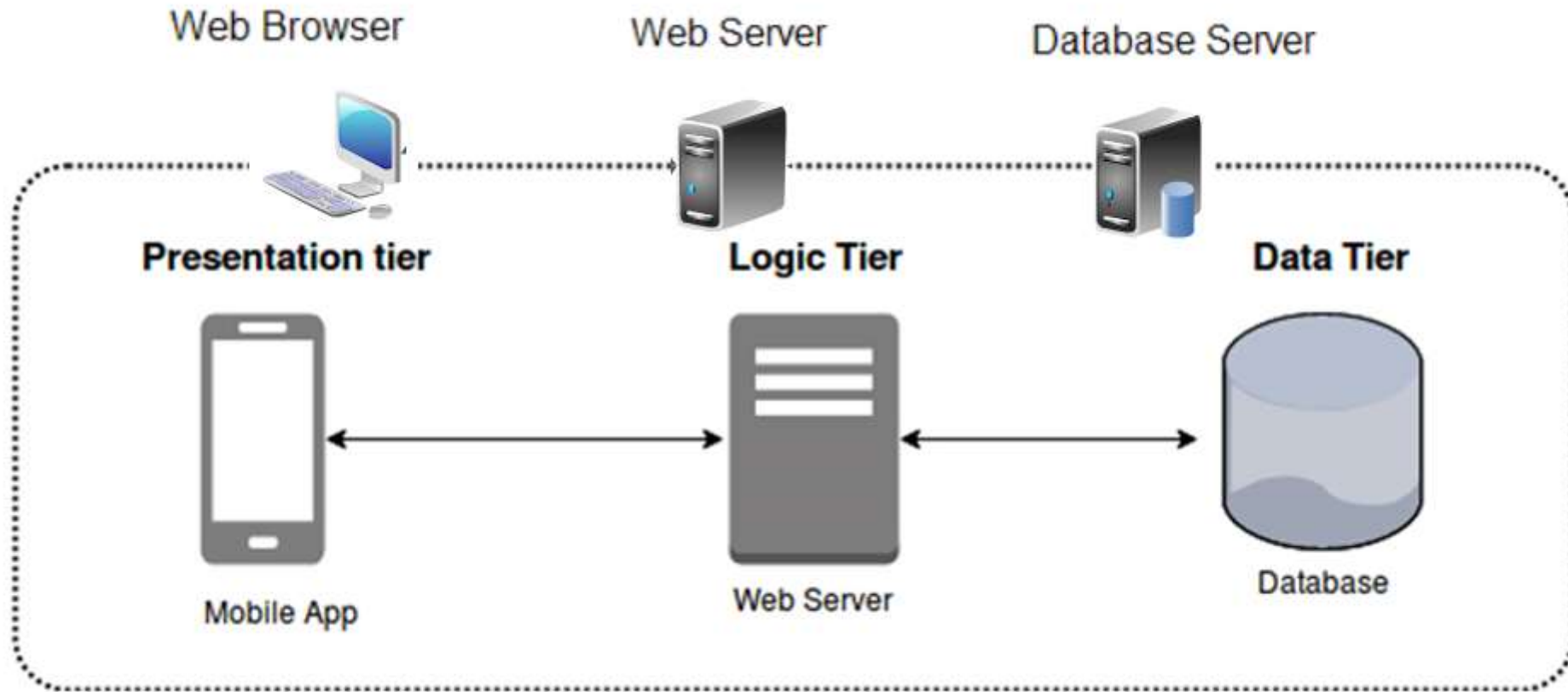
Application Programming Interface

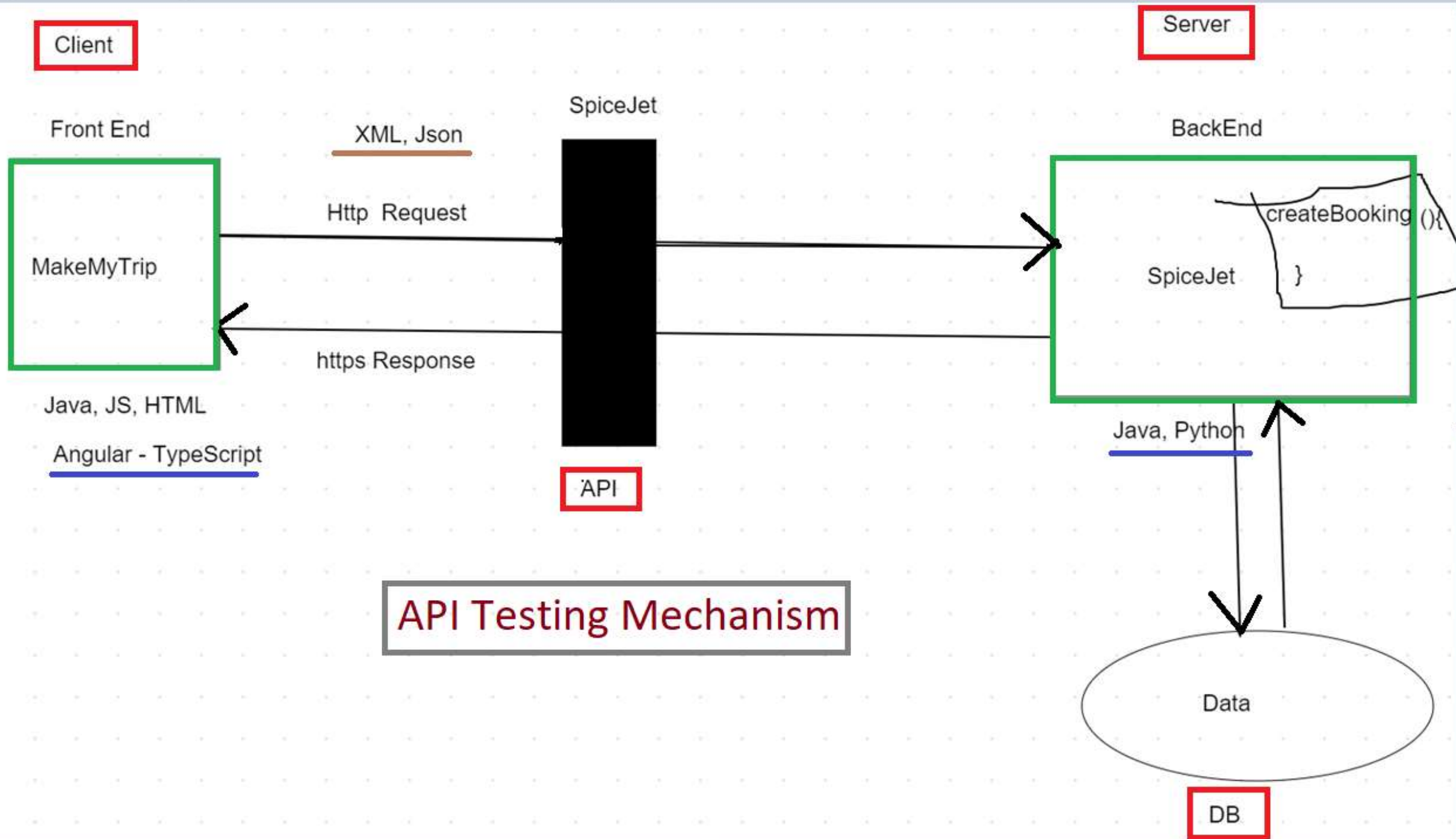


Application Tiers – 3 Layers

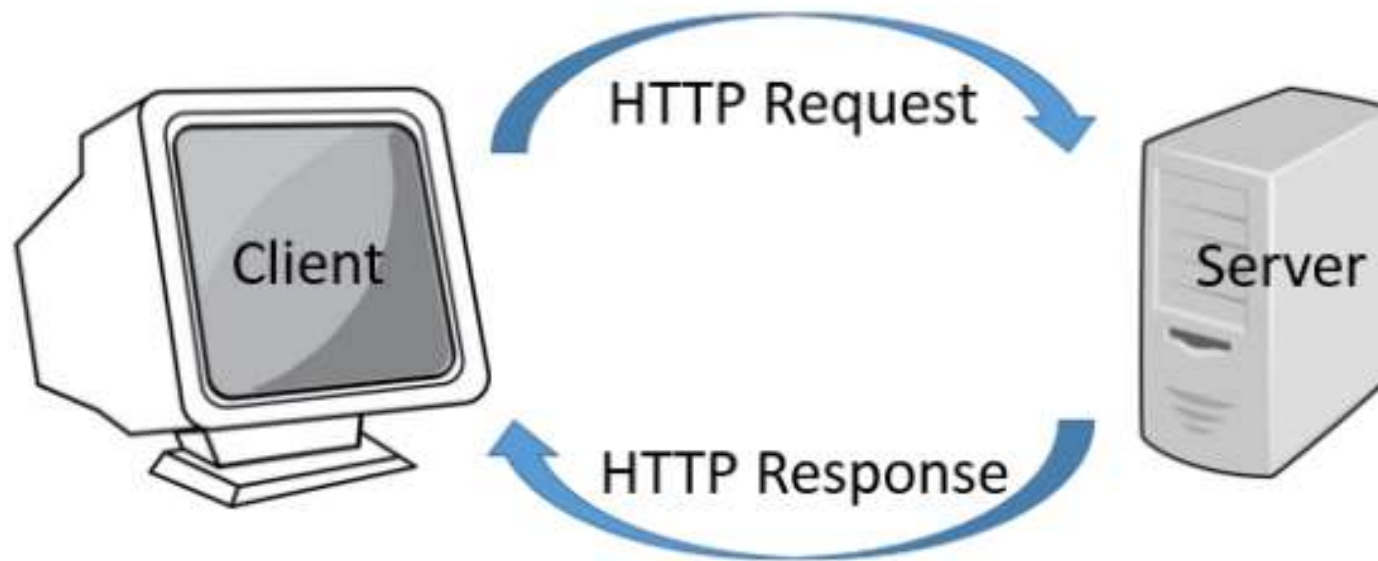


Architecture

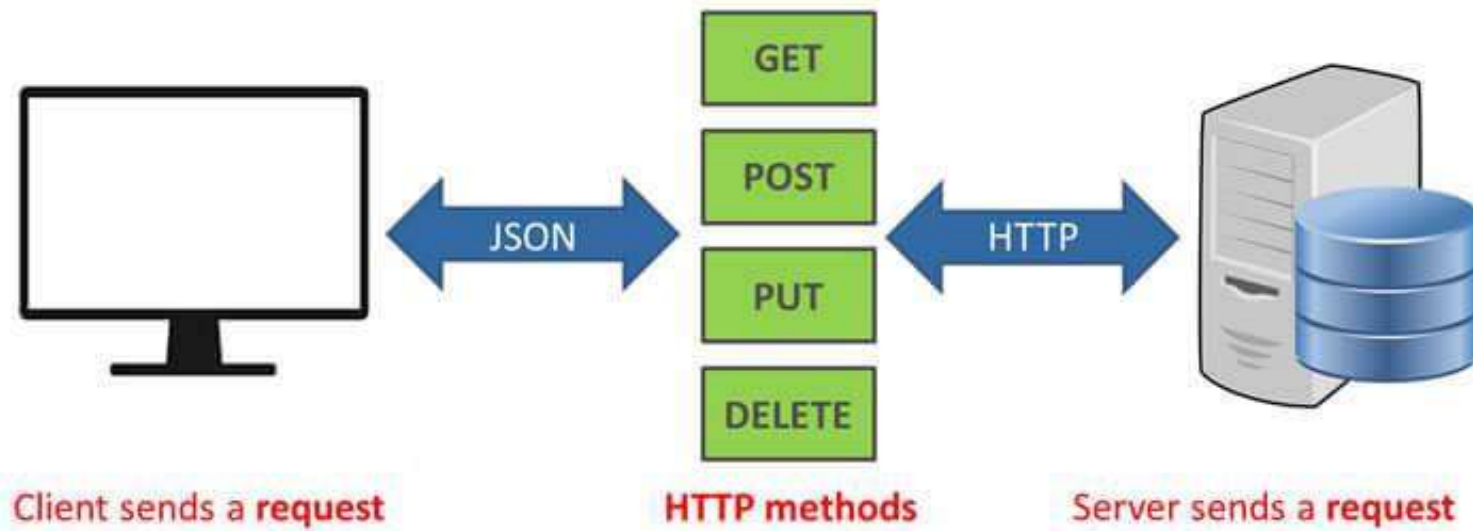




http://



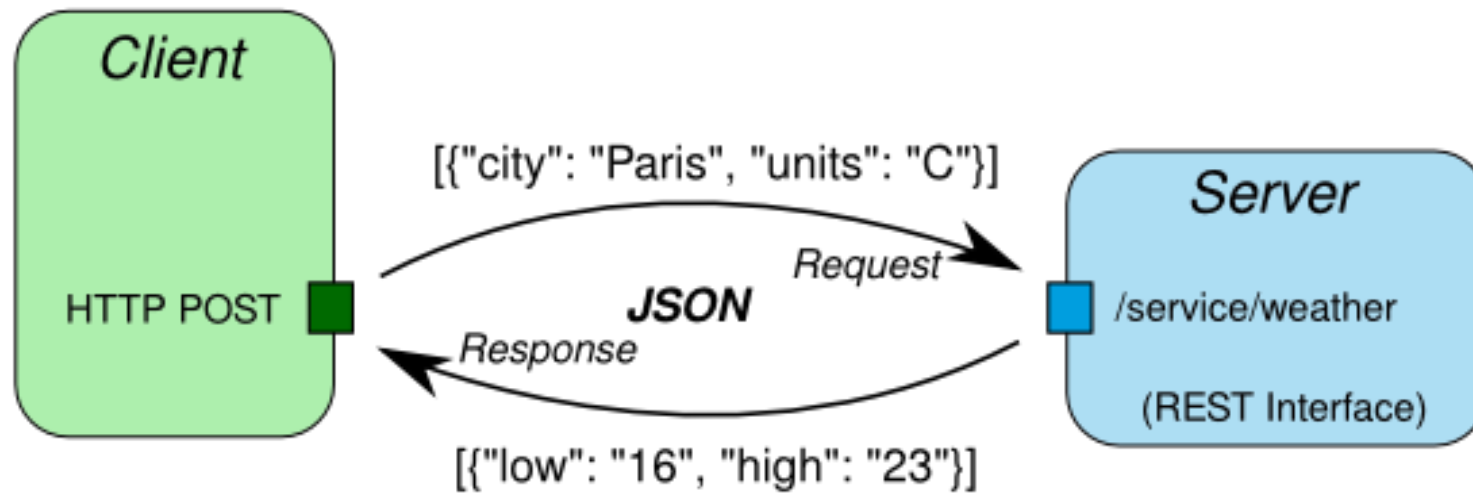
HTTP Methods



HTTP Methods and Their Meaning

POST	→	Create
GET	→	Read
PUT	→	Update
DELETE	→	Delete

JSON / REST / HTTP



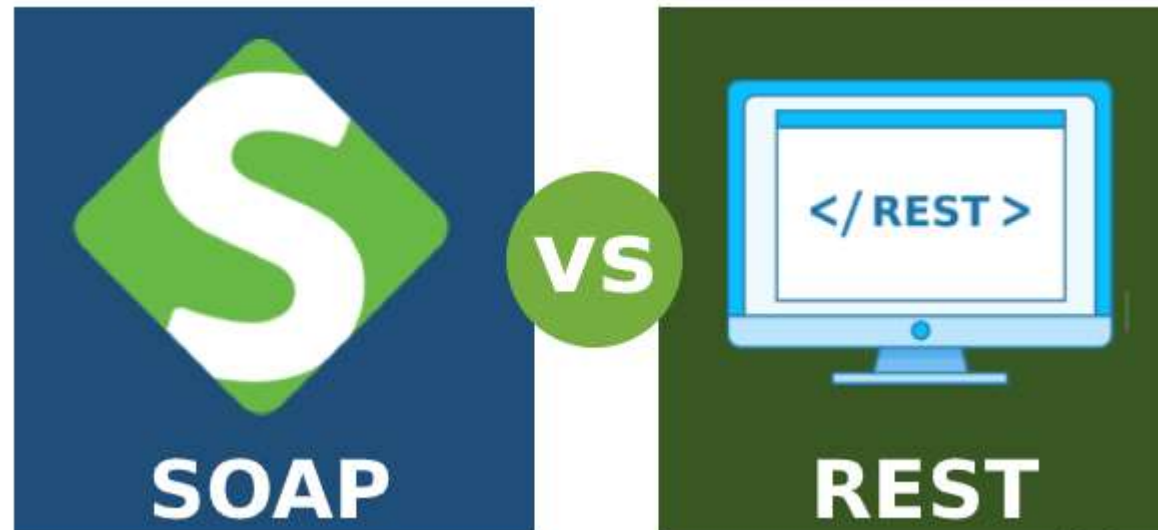
XML

```
<empinfo>
  <employees>
    <employee>
      <name>James Kirk</name>
      <age>40</age>
    </employee>
    <employee>
      <name>Jean-Luc Picard</name>
      <age>45</age>
    </employee>
    <employee>
      <name>Wesley Crusher</name>
      <age>27</age>
    </employee>
  </employees>
</empinfo>
```

JSON

```
{ "empinfo" :
  {
    "employees" : [
      {
        "name" : "James Kirk",
        "age" : 40,
      },
      {
        "name" : "Jean-Luc Picard",
        "age" : 45,
      },
      {
        "name" : "Wesley Crusher",
        "age" : 27,
      }
    ]
  }
}
```

API Types



**Simple Object Access
Protocol**

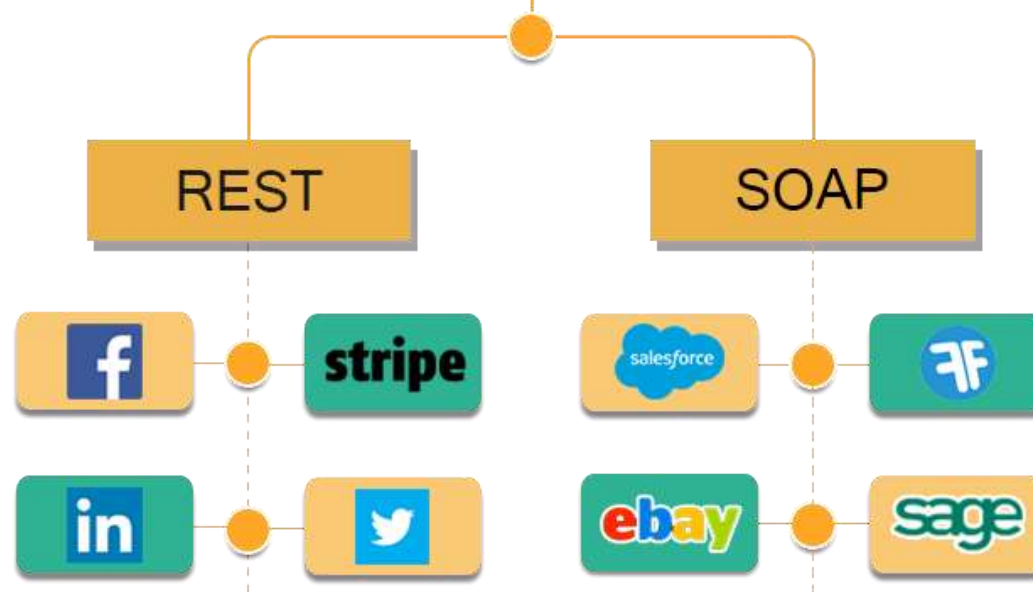
**Representational State
Transfer**

SOAP vs REST

SOAP	REST
• SOAP is a protocol through which two computer communicates by sharing XML • SOAP permits only XML • SOAP based reads cannot be cached • SOAP is like custom desktop application, closely connected to the server • SOAP is slower than REST • It runs on HTTP but envelopes the message	• Rest is a service architecture and design for network-based software architectures • REST supports many different data formats • REST reads can be cached • A REST client is more like a browser; it knows how to standardized methods and an application has to fit. • REST is faster than SOAP • It uses the HTTP headers to hold meta information



APIs integrated by us



API Testing Tools



HTTP Status Codes

Level 200 (Success)

200 : OK

201 : Created

203 : Non-Authoritative
Information

204 : No Content

Level 400

400 : Bad Request

401 : Unauthorized

403 : Forbidden

404 : Not Found

409 : Conflict

Level 500

500 : Internal Server Error

503 : Service Unavailable

501 : Not Implemented

504 : Gateway Timeout

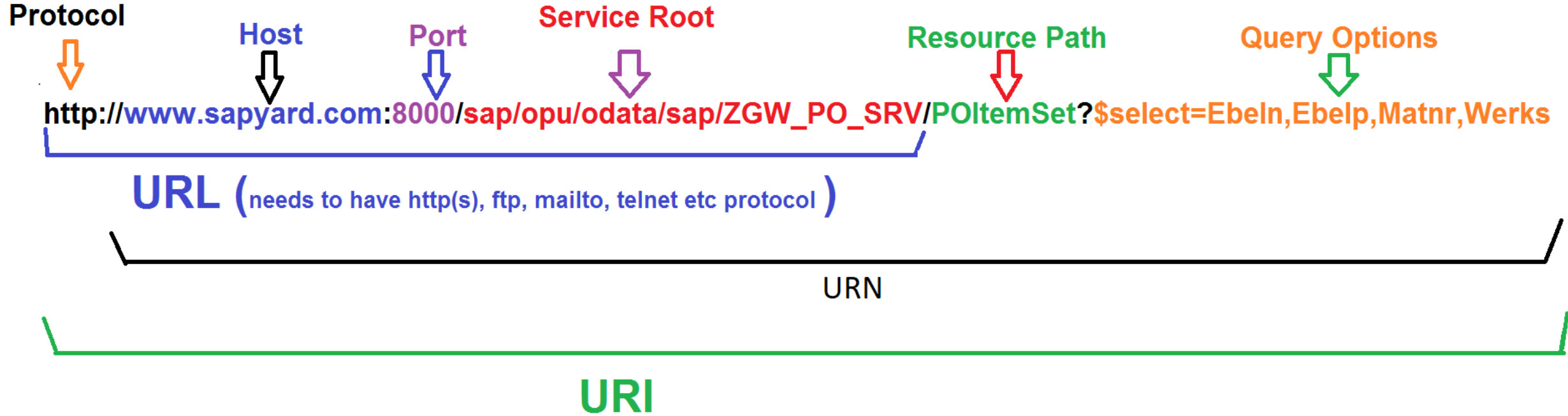
599 : Network timeout

502 : Bad Gateway

URL Structure



URL, URN, URI



URL: Uniform Resource Locator

URN: Uniform Resource Name

URI: Uniform Resource Identifier

Variables



REST API

- Application Programming Interface

It allows software applications to communicate with each other via API Calls

- REST – Representational State Transfer

Ex: Google search, Weather Report.

4 Methods

- GET – Extract data
- POST – Create new
- PUT - Update existing one
- DELETE – Removes all current representations

Interview Questions

Part-1

1. API Documentation - SwaggerUI
2. How to integrate excel with rest assured
3. What is the protocol used in your api testing?
4. What are the protocol methods?
5. How will you do the data validation?
6. Diff JSON Object and JSON Array
7. What is JsonPath? its uses.
8. How will you pass the request?

Ans:- PayLoad (JSON File)
- Can create using POJO class

Interview Questions

Part-2

9. What is Serialization?

Ans: Java Object --> JSON Object
i.e: Creating PayLoad

10. What is De-Serialization?

Ans: JSON Object --> JSON Object
Data validation

11. How will you achieve that?

- Using POJO Class

12. Status code?

13. diff Put & Patch

14. What is the framework?

- Cucumber-BDD

1xx Informational

100 Continue

2xx Success

★ 200 OK

203 Non-Authoritative Information

206 Partial Content

226 IM Used

3xx Redirection

300 Multiple Choices

303 See Other

306 (Unused)

4xx Client Error

★ 400 Bad Request

★ 403 Forbidden

406 Not Acceptable

★ 409 Conflict

412 Precondition Failed

415 Unsupported Media Type

418 I'm a teapot (RFC 2324)

423 Locked (WebDAV)

426 Upgrade Required

431 Request Header Fields Too Large

450 Blocked by Windows Parental Controls (Microsoft)

5xx Server Error

★ 500 Internal Server Error

503 Service Unavailable

506 Variant Also Negotiates (Experimental)

509 Bandwidth Limit Exceeded (Apache)

598 Network read timeout error

101 Switching Protocols

★ 201 Created

★ 204 No Content

207 Multi-Status (WebDAV)

301 Moved Permanently

★ 304 Not Modified

307 Temporary Redirect

★ 401 Unauthorized

★ 404 Not Found

407 Proxy Authentication Required

410 Gone

413 Request Entity Too Large

416 Requested Range Not Satisfiable

420 Enhance Your Calm (Twitter)

424 Failed Dependency (WebDAV)

428 Precondition Required

444 No Response (Nginx)

451 Unavailable For Legal Reasons

501 Not Implemented

504 Gateway Timeout

507 Insufficient Storage (WebDAV)

510 Not Extended

599 Network connect timeout error

102 Processing (WebDAV)

202 Accepted

205 Reset Content

208 Already Reported (WebDAV)

302 Found

305 Use Proxy

308 Permanent Redirect (experimental)

402 Payment Required

405 Method Not Allowed

408 Request Timeout

411 Length Required

414 Request-URI Too Long

417 Expectation Failed

422 Unprocessable Entity (WebDAV)

425 Reserved for WebDAV

429 Too Many Requests

449 Retry With (Microsoft)

499 Client Closed Request (Nginx)

502 Bad Gateway

505 HTTP Version Not Supported

508 Loop Detected (WebDAV)

511 Network Authentication Required

Swagger

- ✓ Swagger is a framework for describing your API by using a common language that is easy read and understand by developers and testers. Its like blueprint for a house.

SwaggerUI

- ✓ Swagger UI, a part of Swagger, is an open source tool that generates a web page that documents the APIs generated by the Swagger specification.
- ✓ This UI presentation of the APIs is user friendly and easy to understand, with all logic complexity kept behind the screen.
- ✓ This enables developers to execute and monitor the API requests they sent and the results they received, making it a great tool for developers, testers and end consumers to understand the end points they are testing.
- ✓ Swagger UI represents apis within browser so brings more intuitively than other

REST API Authentication Methods

1. Inheriting auth
2. No auth
3. API key
4. Bearer token
5. Basic auth
6. Digest auth
7. OAuth 1.0
8. OAuth 2.0
9. Hawk authentication
10. AWS Signature
11. NTLM authentication
12. Akamai EdgeGrid

Multi Factor Authentication

How to choose a automation Framework

Options in API

- ✓ The HTTP **OPTIONS** method requests permitted communication options for a given URL or server.
- ✓ A client can specify a URL with this method, or an asterisk (*) to refer to the entire server.

Syntax:

```
OPTIONS /index.html HTTP/1.1  
OPTIONS * HTTP/1.1
```

The screenshot displays the Postman interface for configuring an HTTP request. The top bar includes tabs for Runner, Import, and Builder (which is active), along with a Team Library link and status indicators (OFFLINE). The address bar shows the URL `http://localhost:8080/` with a dropdown menu set to `Options`. The request URL is `http://localhost:8080/RestFulTutorial/rest/books/`. The `Headers` tab is selected, showing a table with one header: `Content-Type` with the value `text/plain`. The `Body` tab is also visible. The response section at the bottom shows the status `200 OK` and the time `6308 ms`. The response body contains the following information:

- `Allow` → POST, PUT, GET
- `Content-Length` → 0
- `Content-Type` → application/json
- `Date` → Thu, 27 Jul 2017 18:47:08 GMT
- `Server` → Apache-Coyote/1.1

Head in API

- ✓ The HEAD method is similar to the GET method except the message body is not returned in response.
- ✓ The response only includes meta-information, such as a response code or corresponding headers.
- ✓ Checking whether a resource has changed. This is useful when maintaining a cached version of a resource

✓ Retrieve

```
HTTP/1.1 200 OK
Accept-Ranges: bytes
Content-Type: text/html; charset=UTF-8
Date: Wed, 08 May 2013 10:12:29 GMT
ETag: "780602-4f6-4db31b2978ec0"
Last-Modified: Thu, 25 Apr 2013 16:13:23 GMT
Content-Length: 1270
```

making a possibly costly retrieval

Header

- ✓ **HTTP headers** let the client and the server pass additional information with an **HTTP request** or **response**

Example:

- ✓ **Accept-* headers** indicate the allowed and preferred formats of the response

All Headers

The JSON API uses the following standard HTTP headers:

<u>Authorization</u>	<u>Content-Type</u>	<u>Host</u>
<u>Cache-Control</u>	<u>Content-Transfer-Encoding</u>	<u>If-Match</u>
<u>Content-ID</u>	<u>Date</u>	<u>If-None-Match</u>
<u>Content-Length</u>	<u>ETag</u>	<u>Location</u>
<u>Content-Range</u>	<u>Expires</u>	<u>Range</u>

Request Header

- ✓ A client sends request header fields to provide more information about the request context, make the request conditional based on
 - the target resource state,
 - suggest preferred formats for the response,
 - supply authentication credentials, or
 - modify the expected request processing.
- ✓ These fields act as request modifiers, similar to the parameters on a programming language method invocation.

Response Header

- ✓ The response header fields allow the server to pass additional information about the response beyond what is placed in the status- line.
- ✓ These header fields give information about the server, about further access to the target resource, or about related resources.
- ✓ Although each response header field has a defined meaning, in general, the precise semantics might be further refined by the semantics of the request method and/or response status code.

Accept & Content-Type

Accept

The "Accept" header field can be used by user agents to specify **response** media types that are acceptable.

Content-Type

The "Content-Type" header field indicates the media type of the associated representation

Payload

The **Payload** of an **API** Module is the body of your request and response message. It contains the data that you send to the server when you make an **API** request.

You can send and receive **Payload** in different formats, Ex., JSON.

Cache vs Cookies

Cache

It is used to store online page resources during a browser for the long run purpose or to decrease the loading time.

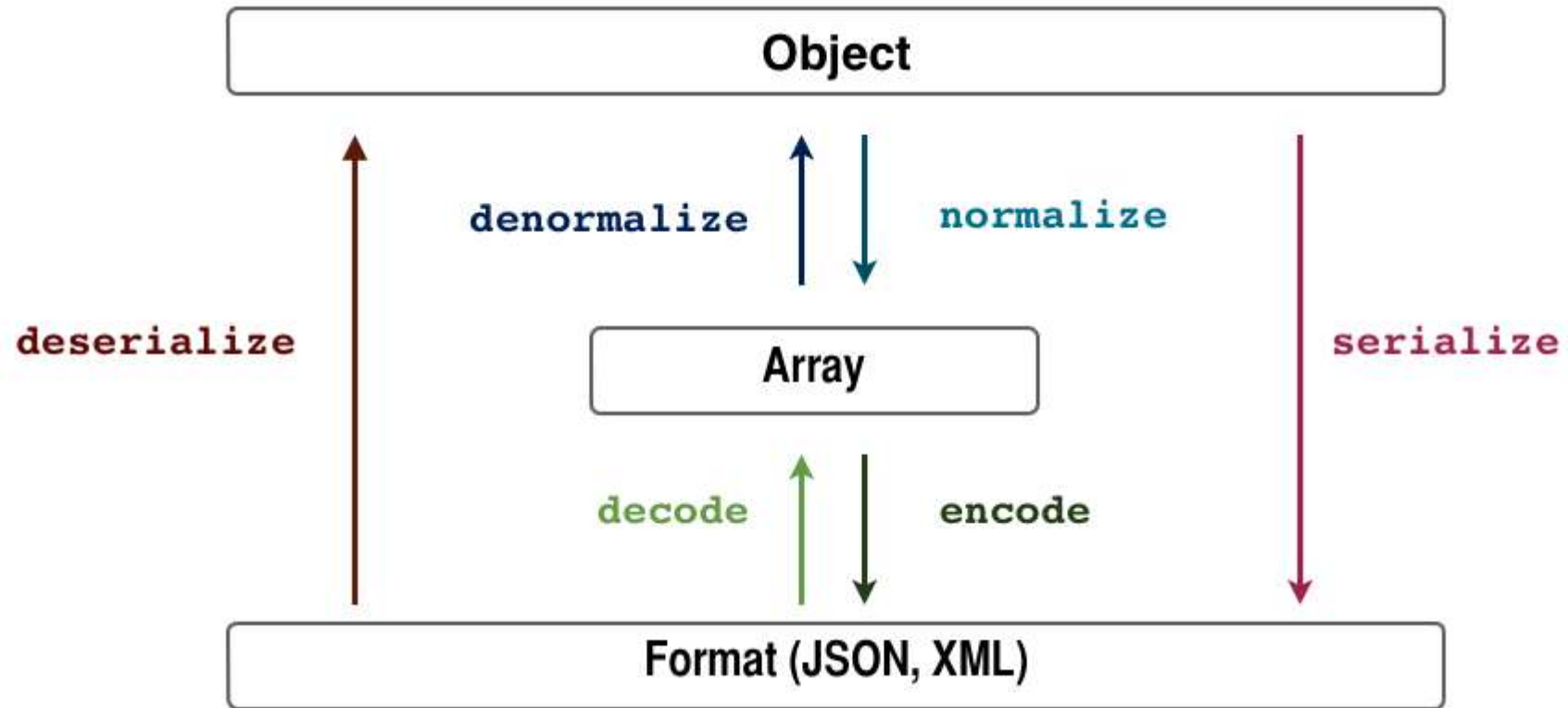
It consumes large space in terms of capacity.

Cookies

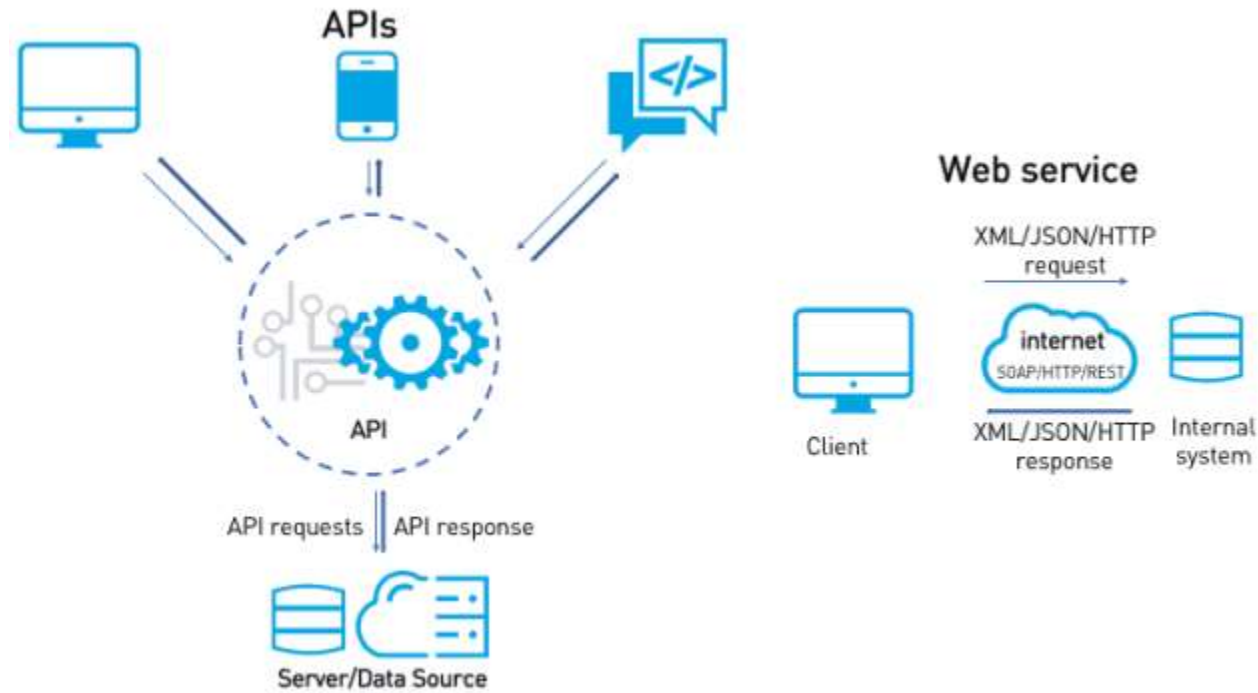
are employed to store user choices such as browsing session to trace the user preferences.

While it consumes less space in terms of capacity.

Serialization vs Deserialization



API Vs Web Services



Parameter in API

Testcase

TCID	Description	EndPoint	HTTP Method	URI	Body	Success Response	Status Code	Comments
1	Weather API - Validate status code & status line	http://restapi.demoga.com/utilities/weather/city/	GET	http://restapi.demoga.com/utilities/weather/city/Hyderabad	NA	{ "City": "Hyderabad", "Temperature": "31.49 Degree celsius", "Humidity": "62 Percent", "Weather Description": "scattered clouds", "Wind Speed": "3.6 Km per hour", "Wind Direction degree": "270 Degree" }	200	Here <city> means the city for which we are trying to retrieve the weather data. For Eg: If you want to know the weather conditions of Hyderabad, you would simply replace the City text with Hyderabad
2	Register Customer API	http://restapi.demoga.com/customer/register	POST	http://restapi.demoga.com/customer/register	{ "FirstName": "value", "LastName": "value", "UserName": "value", "Password": "value", "Email": "Value" }	{ "SuccessCode": "OPERATION_SUCCESS", "Message": "Operation completed successfully" }	201	NA
3	Google Map API - Validating Headers	https://maps.googleapis.com/	GET	https://maps.googleapis.com/maps/api/place/nearbysearch/xml?location=33.8670522,151.1957362&radius=1500&type=supermarket&key=AIza5yB6GCE3VplU4lgTqSTDmHm/2HoE1b4/y1s	NA	Returns list of super markets	200	Headers:
4	Google Map API - Printing all Headers	https://maps.googleapis.com/	GET	https://maps.googleapis.com/maps/api/place/nearbysearch/xml?location=33.8670522,151.1957362&radius=1500	NA	Returns list of super markets	200	Printing ALL Headers

API Testing Challenges

1. Initial Setup of API Testing
2. Updating the Schema of API Testing
3. Testing Parameter Combinations
4. Sequencing the API Calls
5. Validating Parameters

Questions

1. Can we use GET request instead of PUT to create a resource?
2. Idempotent

